

PROJETO E ANÁLISE DE ALGORITMOS – 01A – 2022.2

[Página inicial](#)

[Meus cursos](#)

[PROJETO E ANÁLISE DE ALGORITMOS – 01A – 2022.2](#)

[Tópico 7](#)

[Notas de aula: solução de recorrências: Teorema Mestre](#)

Recorrências

Uma **recorrência** descreve uma função $f(n)$ utilizando o resultado de f aplicada em valores menores que n . Quando um algoritmo tem chamadas recursivas, seu tempo de execução é descrito por recorrência.

Como encontrar a complexidade de um algoritmo recursivo? Ou seja, como determinar o comportamento assintótico de uma recorrência? Hoje veremos como resolver um tipo comum de recorrência utilizando o Teorema Mestre. As 3 primeiras páginas de [recorrencias-4up.pdf](#) falam sobre recorrências, e na última página temos a definição do Teorema Mestre. No Cormen, este assunto aparece na Seção "Método mestre para resolver recorrências", do Capítulo "Divisão e conquista". O Vazirani faz uma apresentação curta do assunto na Seção "Relações de recorrência" do Capítulo "Algoritmos de divisão-e-conquista".

Teorema Mestre

Permite resolver rapidamente várias recorrências do tipo $T(n) = aT(n/b) + f(n)$, que são comuns em algoritmos de divisão e conquista (como o MergeSort).

Projeto de algoritmos por divisão e conquista:

Divisão: divida o problema em subproblemas menores.

Conquista: resolva os subproblemas recursivamente. Se o tamanho do subproblema é suficientemente pequeno, resolva-o diretamente (sem recursão).

Combinação: utilize a solução dos subproblemas para produzir a solução do problema original.

Teorema Mestre:

Seja $T(n) = aT(n/b) + f(n)$, com $a \geq 1, b > 1$ e $f(n)$ assintoticamente positiva (converge para valor positivo ou $+\infty$).

Caso 1: Se $f(n) \in O(n^{\log_b a - \epsilon})$ p/ alguma constante $\epsilon > 0$, então $T(n) \in \Theta(n^{\log_b a})$.

Caso 2: Se $f(n) \in \Theta(n^{\log_b a})$, então $T(n) \in \Theta(n^{\log_b a} \lg n)$.

Caso 3: Se $f(n) \in \Omega(n^{\log_b a + \epsilon})$ p/ alguma constante $\epsilon > 0$, e $a \cdot f(n/b) \leq c \cdot f(n)$ p/ alguma constante $c < 1$, então $T(n) \in \Theta(f(n))$.

A condição " $a \cdot f(n/b) \leq c \cdot f(n)$ p/ alguma constante $c < 1$ " é chamada **condição de regularidade** do caso 3. As funções $f(n)$ que já observei em análise de algoritmos não violam a condição de regularidade. Como exemplo de recorrência que atende as condições do caso 3, exceto a condição de regularidade, temos

$$T(n) = T(n/2) + n(2 - \cos n).$$

◀ [Vídeo: demonstração das regras de aproximação de recorrências \(18min\)](#)

Seguir para...

[Notas de aula: solução de recorrências: método da substituição ►](#)

 Baixar o aplicativo móvel.